

Agentic RAG

A complete end-to-end walkthrough of every layer: clients → FastAPI → runner → memory → the seven agent modes → retrieval → generation → production envelope. Written from the live code paths so you can re-derive the system without guessing.

What this document is for. Keep it beside the repo. Every major step names the function, file, and decision. If code and this manual diverge, trust the code — then update this file.

Edition: August 2026 (deep-dive revision)

Scope: Full stack — agents, memory, retrieval, API, frontend, ops

Default mode: `agentic`

Canonical entry: `run_agent()` / `stream_agent()`

Stack: LangChain · LangGraph · FastAPI · Chroma · OpenAI · Redis · React · Supabase

Contents

1. How to read this document
2. System at a glance
3. Layered architecture
4. End-to-end request lifecycle (every step)
5. Conversation memory (core)
6. The AI agent system (all 7 modes)
7. Decision chains the agents use
8. Ingestion & retrieval pipeline
9. Generation, citations, follow-ups, streaming
10. Production envelope
11. API surface, clients & deploy
12. Configuration map
13. File ownership quick reference
14. Optional features & known gaps
15. Tests that document behavior
16. Worked traces

1. How to read this document

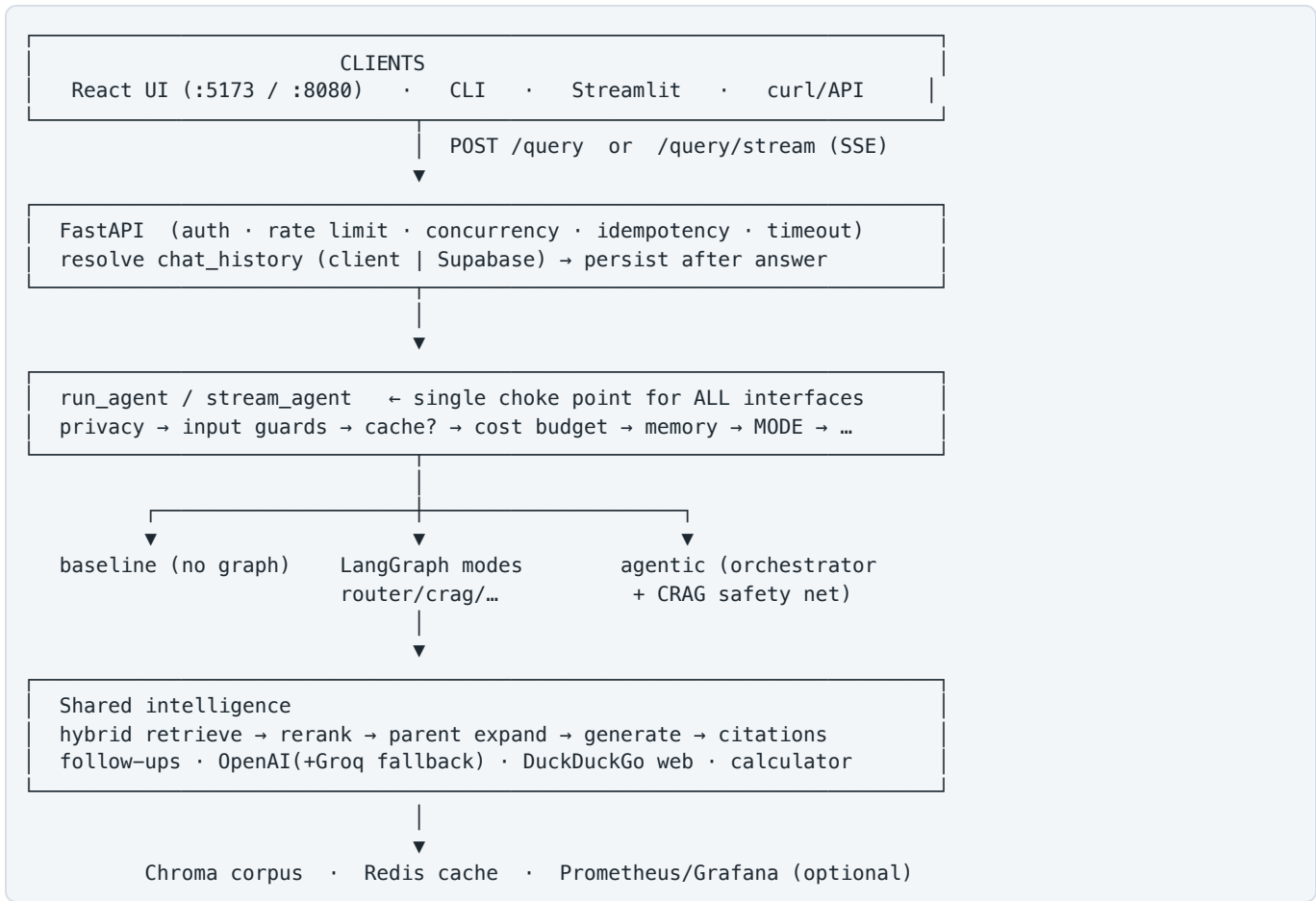
Start with §2–§4 for the whole picture. Read §5 carefully — memory is a first-class product feature. Spend the most time on §6–§7 for the agent system. Use §8–§11 when debugging ingestion, retrieval, or production. §16 has concrete CLI walkthroughs.

If you need...	Go to
One diagram of the whole product	§2 System at a glance
What happens on every API/CLI call, step by step	§4 Request lifecycle
How conversation history is packed and persisted	§5 Memory
How each agent mode thinks and routes	§6 AI agent system
Why agentic picked decompose vs multi-hop	§6.7 and §7
How PDFs become searchable chunks	§8 Ingestion & retrieval
Auth, rate limits, cache, metrics, Docker	§10–§11
What is optional vs always on	§14 Gaps

STABILITY NOTE
Mode keys (`baseline` ... `agentic`), graph node names, and the `run_agent` choke point are the durable contracts. Prompt text and default model names may change; treat them as tunable, not structural.

2. System at a glance

Agentic RAG is a **multi-mode research assistant**. One question can be answered seven different ways — from a fixed retrieve→generate baseline up to a full orchestrator that chooses strategy, grades evidence, and may rewrite or fall back to the web. Every path shares the same retrieval stack, citations, guardrails, memory packing, and optional Redis cache.



Mode key	Phase	One-line behavior
baseline	1	Always retrieve → generate. No decisions.
router	2	Route to direct / retrieve / web.
crag	3	Grade docs; rewrite & retry; else web fallback.
decompose	4	Split into sub-queries; retrieve in parallel; synthesize.
multi_hop	5	Sequential hops; each query depends on prior findings.
tools	6	Function-calling loop: retrieve / web / calculator.
agentic	7	Router → strategy picker → subgraph → grade → answer.

3. Layered architecture

Think in five layers. Higher layers never bypass lower safety layers.

Layer	Responsibility	Primary modules
1. Interface	HTTP, CLI, UI. Auth headers, SSE framing, session IDs.	src/api/server.py, frontend/, src/cli.py, streamlit_app.py
2. Control plane	Privacy, input/output guards, cost budgets, cache, memory packing.	runner.py, guardrails.py, privacy.py, cache/, memory/
3. Agent graphs	Mode-specific LangGraph StateGraphs (or baseline pipeline).	src/graph/*, src/rag/baseline.py, src/agents/*
4. Knowledge plane	Ingest, hybrid search, rerank, parent expansion, citations.	src/ingestion/*, src/retrieval/*
5. Model & tools	LLM client, structured outputs, tools, web search.	src/llm.py, src/chains/, src/tools/

HARD RULE
Never call ask_baseline, ask_crag, etc. from a new interface. Always go through run_agent() or stream_agent(). Skipping the runner skips privacy, rate/cost guards, cache, memory augmentation, and follow-ups.

4. End-to-end request lifecycle (every step)

This is the spine of the system. Below is the exact order of operations for a successful POST /query, then the streaming variant, then how CLI/Streamlit differ.

4.0 Master flow

```
Client
  → middleware (TrustedHost? → RequestId → CORS)
  → verify_api_key + enforce_client_rate_limit
  → (sync only) Idempotency-Key lookup
  → _resolve_chat_history (client history | Supabase | none)
  → concurrency semaphore
  → run_agent / stream_agent
    A. PrivacyGuard.apply_input
    B. InputGuardrails.validate
    C. should_use_cache?
    D. augment_question_with_history (if memory + history)
    E. cache hit? → post-guards → return
    F. CostGuardrails (QPM + token budgets) + record_query
    G. _dispatch(mode) → graph / baseline
    H. OutputGuardrails + PrivacyGuard.apply_output
    I. optional QualityGuardrails
    J. generate_follow_ups
    K. set_cached_response (if eligible)
  → _persist_exchange (Supabase, if session)
  → QueryResponse / SSE frames + metrics
```

4.1 FastAPI bootstrap

1. **Entry:** `uvicorn src.api.server:app` (or `python -m src.api.server`).
2. **Import:** `import src.bootstrap` initializes LangSmith env before LangChain imports.
3. **Lifespan:** `setup_logging()`, `init_langsmith_tracing()`, deprecation warnings, and `— if is_production()` `— _validate_production_config()`.
4. **Production refuse-to-start when:** missing `OPENAI_API_KEY`; auth required but no `API_KEY`; `CORS_ORIGINS=*`; `API_KEY` shorter than 32 chars; `API_WORKERS>1` with `RATE_LIMIT_BACKEND=memory`.

4.2 Middleware (order)

In Starlette, the *last* middleware added runs *first* on the request. Effective inbound order:

1. `TrustedHostMiddleware` — only if `TRUSTED_HOSTS` is set.
2. `RequestIdMiddleware` — accept or mint `X-Request-ID`, bind via `set_request_id()`, echo on response.
3. `CORSMiddleware` — origins from `CORS_ORIGINS`; credentials only when origin is not `*`; methods `GET|POST`; headers include `X-API-Key`, `Idempotency-Key`.

4.3 Request models

Model	Fields that matter
QueryRequest	question (1–2000), mode (default agentic), session_id (8–64 [A–Za–z0–9_–]+), use_memory (default true), chat_history (≤ 20 turns)
ChatTurnIn	role $\in$ {user, assistant}, content 1–8000
QueryResponse	answer, sources, citations, route*, steps, follow_ups, latency_ms, session_id, error_code
CitationOut	index, chunk_id, source, page, section, snippet, score

4.4 Synchronous path — `POST /query`

1. **Dependencies:** `verify_api_key + enforce_client_rate_limit`.
2. **Idempotency (optional):** if `Idempotency-Key` present $\rightarrow$ `get_idempotent_response`. Body-hash mismatch $\rightarrow$ **409**. Hit $\rightarrow$ return stored `QueryResponse` with metric status `idempotent`.
3. **History:** `_resolve_chat_history(session_id, use_memory, client_history)` (see §5).
4. **Concurrency:** acquire `_query_semaphore` (size `max_concurrent_queries`) within `concurrency_acquire_timeout_seconds`; else **503** + `Retry-After: 5 + record_capacity_rejection()`.
5. **Execute:** `_run_agent_with_timeout = asyncio.to_thread(run_agent, ...)` + `asyncio.wait_for(request_timeout_seconds)`.
6. **Persist:** `_persist_exchange(session_id, question, answer, mode)` when memory + Supabase are active.
7. **Respond:** build `QueryResponse`; store idempotent response if keyed; `record_request(mode, "query", status, latency)`.

Failure	HTTP
TimeoutError	504
RateLimitError (process budget)	429 + Retry-After 60
ValueError (guards / privacy block / bad mode)	400
Unhandled	500 (no internal detail leaked)

GAP

Idempotency applies only to `/query`, not `/query/stream`.

4.5 Streaming path — `POST /query/stream`

1. Same auth + per-client rate limit (no idempotency).
2. Resolve chat history.
3. Spawn a worker thread that runs `stream_agent(..., cancelled=threading.Event)`.
4. Events are pushed onto an `asyncio.Queue` via `loop.call_soon_threadsafe`.
5. Under the concurrency slot, pull events with dual timeouts:
 - **Gap:** `request_timeout_seconds` between events
 - **Wall:** `stream_timeout_seconds` total deadline
6. On `type==answer`: capture text for later persist.
7. On `type==done`: persist exchange; inject `latency_ms` + `session_id`; metric `ok`.
8. Yield SSE frames: `data: {json}\n\n`.
9. On error / timeout / client disconnect: `cancelled.set()`, cancel producer.

SSE <code>type</code>	Meaning
<code>step</code>	Agent/graph progress line (router decision, hop, grade, ...)
<code>token</code>	Final-answer token as it generates
<code>answer</code>	Full answer text (after generation / post-guards)
<code>follow_ups</code>	List of follow-up questions
<code>sources</code>	Source labels + <code>citations</code> array
<code>done</code>	Terminal frame: latency, session, mode, route, steps; may include <code>cached: true</code>
<code>error</code>	Guardrail / runtime failure

Response headers: `Cache-Control: no-cache`, `Connection: keep-alive`, `X-Accel-Buffering: no`. Media type: `text/event-stream`.

4.6 Inside `run_agent()` — the choke point

File: `src/runner.py`. Every interface that wants correct behavior calls this (or `stream_agent`).

A. `_prepare_agent_run`

1. `PrivacyGuard.apply_input(question, policy)` — block → `ValueError`; else redacted text as `sanitized`.

2. `InputGuardrails.validate(sanitized)` — length, word count, credential-shaped strings (not a keyword blacklist).
3. `should_use_cache(use_memory, chat_history)` — false when cache disabled *or* memory is on with non-empty history (history would change the answer).
4. If `use_memory` and `chat_history`: `effective_question = augment_question_with_history(sanitized, chat_history)`.
5. Else: `effective_question = sanitized`.
6. Return `_Preflight: sanitized (cache key), effective (model input), tracker, cacheable flag`.

WHY BUDGET IS NOT HERE

Cost/rate budgets are deliberately *not* consumed in prepare, so cache hits are free.

B. Cache + budget

1. If cacheable: `get_cached_response(sanitized, mode)` → on hit, run `_apply_post_guardrails` and return (no LLM).
2. `_consume_budget(tracker): check_query_rate + check_token_budget → RateLimitError; then record_query()`.
3. `_run_with_cost_tracking(effective_question, mode, tracker):`
 - `reset_llm_provider()`
 - Prefer OpenAI callback around `_dispatch`; Groq fallback estimated if needed
 - `_record_spend` → tracker + Prometheus

C. `_dispatch(question, mode)`

mode	entrypoint
baseline	<code>src.rag.baseline.ask_baseline</code>
router	<code>src.graph.router_graph.ask_router</code>
crag	<code>src.graph.crag_graph.ask_crag</code>
decompose	<code>src.graph.decompose_graph.ask_decompose</code>
multi_hop	<code>src.graph.multi_hop_graph.ask_multi_hop</code>
tools	<code>src.graph.tools_graph.ask_tools</code>
agentic	<code>src.graph.agent_graph.ask_agentic</code>
else	<code>ValueError</code>

D. Post-processing

1. `_apply_post_guardrails: OutputGuardrails.validate (warn); PrivacyGuard.apply_output (block → ValueError); replace answer`.
2. `_finalize_agent_result:`
 - If `error_code` (e.g. `node_gate_abort`): clear follow-ups, **no cache**, return.
 - Else optional `_maybe_quality_check` (only if `QUALITY_GUARDRAILS_ENABLED=true`).
 - `_attach_follow_ups` → `generate_follow_ups` (never fails the request).

- If cacheable: `set_cached_response` (serializable fields; not full `context_docs`).

4.7 `stream_agent()` differences

- Prep errors yield `{type: "error"}` instead of raising through HTTP mapping alone.
- Cache hit emits synthetic `step / answer / follow_ups / sources / done(cached:true)` so the UI path is identical.
- Otherwise: worker + `use_emitter(emit)` ; client disconnect sets `CancelledRun`.
- During run: steps/tokens via emitter; after: `answer` → `follow_ups` → `sources` → `done` (+ `error_code` if abort).

4.8 How non-HTTP clients enter

Client	Entry	Skips FastAPI layers?
React UI	<code>streamQuery</code> → <code>POST /query/stream</code>	No — full API envelope
curl / API	<code>POST /query</code> or <code>stream</code>	No
Streamlit	in-process <code>run_agent</code>	Yes — no HTTP auth/rate/concurrency/idempotency
CLI	<code>python -m src.cli ask ...</code> → <code>run_agent</code>	Yes — no memory/session by default

5. Conversation memory (core)

Memory is not a side feature. Follow-up questions (“what about the second approach?”) only work because prior turns are packed into the effective question before any mode runs.

5.1 Two layers

Layer	Module	Role
Prompt packing	src/memory/chat_memory.py	Always available. Compacts history into the question text.
Persistence	src/memory/supabase_store.py	Optional. Cross-session load/save by <code>session_id</code> .

5.2 Activation gates

1. Client sends `use_memory` (default `true` in API + React).
2. Server requires `settings.memory_enabled` (env `MEMORY_ENABLED`, default `true`).
3. `_memory_active = use_memory and settings.memory_enabled`.
4. If inactive → history is ignored; no Supabase load/save.

5.3 Resolving history — `_resolve_chat_history`

1. If memory inactive → `(None, session_id)`.
2. Else if client sent `chat_history` → use it (React local chat store path).
3. Else if Supabase configured:
 - With `session_id` → `load_messages(session_id)`
 - Without → mint a new UUID session id and start empty
4. Else → `(None, session_id)` (in-session only via client).

5.4 Compact packing algorithm

Functions: `pair_exchanges` → `format_chat_history` → `augment_question_with_history`.

1. Pair messages into (user_question, assistant_answer|None) exchanges.
2. **Recent N exchanges** (`MEMORY_RECENT_EXCHANGES`, default 3): keep full question + answer truncated to `MEMORY_ANSWER_MAX_CHARS` (default 500).
3. **Older exchanges**: keep questions only (topic trail), capped by `MEMORY_MAX_OLDER_QUERIES` (default 10).
4. Wrap into:

```
Previous conversation:
{history}

Current question: {question}

Instructions:
- If this is a follow-up, use the conversation above for context.
- Prefer recent Q/A pairs for grounding; use earlier questions only for topic continuity.
- If the question stands alone, answer it directly.
```

CACHE INTERACTION

When memory is on *and* history is non-empty, `should_use_cache` returns false. Identical questions in different conversations must not share a cached answer.

5.5 Persistence — Supabase

- Configured when `SUPABASE_URL` + `SUPABASE_KEY` are set.
- `save_message(session_id, role, content, mode)` inserts into `SUPABASE_CHAT_TABLE` (default `chat_messages`).
- `load_messages(session_id, limit=50)` returns chronological role/content.
- Schema: `docs/supabase_schema.sql`.
- API persists after a successful answer via `_persist_exchange` (user then assistant).
- Streamlit can also persist when configured; otherwise it keeps history in `st.session_state.messages`.

5.6 Frontend wiring

- `frontend/src/hooks/useChat.ts`: `useMemory` defaults to `true`.
- Prior turns from the local chat store are sent as `chat_history`.
- `sessionId` is sent when memory is on so the server can persist if Supabase is available.
- `frontend/src/api/client.ts` maps to JSON fields `use_memory` / `chat_history` / `session_id`.

6. The AI agent system (all 7 modes)

Each mode is a different control policy over the same knowledge plane. Node names match the compiled LangGraph graphs in `src/graph/`. Shared helpers: `run_graph_streaming` (`src/streaming.py`) and `build_response` (`src/retrieval/citations.py`).

What every mode returns

An `AgentResponse`: `answer`, `mode`, `sources`, `citations`, `steps`, optional `route` / `grade_summary` / `sub_queries`, and `follow_ups` (attached by the runner).

How to choose a mode

Use `baseline` / `router` for speed; `crag` for quality; `decompose` / `multi_hop` for structure; `tools` for mixed tools; `agentic` when you want the system to pick.

MODE 6.1 baseline — Fixed pipeline

File: `src/rag/baseline.py` · **Not a LangGraph.** Always retrieves, always generates. The control group for every agentic mode.

```
ask_baseline(question)
  → retrieve(question)
  → format_docs(docs)
  → stream_text(rag_chain, {context, question})
  → build_response(mode="baseline", docs=...)
```

Decisions: none. **Use when:** simple facts; latency-sensitive; A/B baseline.

MODE 6.2 router — Intent routing

File: `src/graph/router_graph.py` · **Entry:** `ask_router` · **Decision:** `classify_query` → `direct` | `retrieve` | `web_search`

```
START → classify ┌─ direct ─────────→ direct_answer → END
                  │─ retrieve ─→ retrieve → generate → END
                  └─ web_search → web_search ─────────→ END
```

Nodes:

```
classify_node      → RouteType via structured output
direct_answer_node → direct_chain
retrieve_node      → retrieve()
generate_node      → rag_chain
web_search_node    → DuckDuckGo + web_search_chain
```

Gap vs CRAG/agentic: no document grading, no `node_gate` abort path.

Use when: greetings should skip retrieval; newsy questions need the web.

MODE 6.3 crag — Corrective RAG

File: `src/graph/crag_graph.py` · Adds a **grader loop** after retrieval. Largest quality jump over baseline.

```

START → classify ┤ direct → direct_answer → (abort|END)
                  ├── web_search → web_fallback → (abort|END)
                  └─ retrieve → grade ┤ filtered docs? → generate → (abort|END)
                                      ├── retries left? → rewrite → retrieve (loop)
                                      └─ else → web_fallback → (abort|END)

abort → END

```

Defaults: MAX_RETRIEVAL_RETRIES=2 · GRADER_RELEVANCE_THRESHOLD=0.5
 Abort → error_code="node_gate_abort"

Decision	Function
Route	route_condition (+ abort if gate)
After grade	grade_condition : docs → generate; else rewrite or web_fallback
After leaf	after_node_condition

Use when: answer quality matters and you can afford grading latency.

MODE 6.4 decompose — Parallel sub-queries

File: src/graph/decompose_graph.py · Map-reduce via LangGraph Send to retrieve_sub workers.

```

START → classify ┤ direct → direct_answer → END
                  ├── web → web_search → END
                  └─ retrieve path → decompose
                        ┤ Send(sub_q1) → retrieve_sub
                        ├── Send(sub_q2) → retrieve_sub
                        └─ Send(sub_qN) → retrieve_sub ┤ → synthesize → END

```

Nodes:

- decompose_node → decompose_query → sub_queries
- fan_out_to_retrieve → Send per sub-query
- retrieve_sub_node → retrieve for one sub-query
- synthesize_node → synthesis_chain

Use when: “Compare X vs Y vs Z”, multi-entity questions known upfront.

MODE 6.5 multi_hop — Sequential reasoning

File: src/graph/multi_hop_graph.py · Unlike decompose, hop $N+1$ depends on findings from hop N .

```

START → classify ┤ direct → direct_answer → END
                  ├── web → web_search → END
                  └─ analyze → retrieve_hop → reflect ┤ sufficient → synthesize → END
                                                         └─ next hop (until MAX_MULTI_HOP_STEPS)

```

Helpers: analyze_question, reflect_on_hop (src/agents/multi_hop.py)
 reflect_condition:
 sufficient OR (not multi-hop and hop≥1) OR hop ≥ max → synthesize
 else → retrieve_hop

Use when: “What fallback does CRAG use when retrieval fails?” — must learn CRAG first.

MODE 6.6 tools — Tool-calling agent

File: `src/graph/tools_graph.py` · Node `tools_agent` runs an internal loop (≤ 10 turns) with `llm.bind_tools(TOOLS)`.

```
START → classify ─┬─ direct → direct_answer → END
                  │├─ web → web_search → END
                  └┬─ tools_agent ↻ (tool_calls) → END
                  abort → END

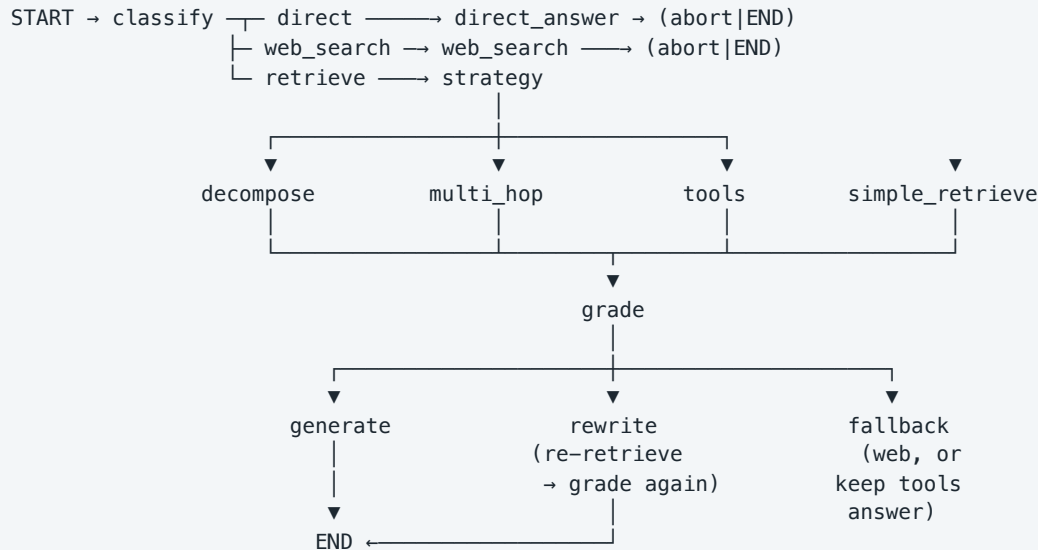
TOOLS (src/tools/all_tools.py):
• retrieve_docs(query) → vector corpus
• web_search(query)    → DuckDuckGo (circuit-breaker wrapped)
• calculator(expr)     → AST-restricted arithmetic (no eval())

check_tool_result quarantines bad outputs;
abort after MAX_TOOL_FAILURES (3).
```

Use when: mixed modalities — math + corpus, or dynamic tool choice.

MODE 6.7 agentic — Full orchestrator (default)

File: `src/graph/agent_graph.py` · **Entry:** `ask_agentic` → `build_full_agent_graph`. Combines routing, strategy selection, the four retrieval strategies, and a Crag-style grade/rewrite/fallback safety net on *every* retrieval path.



Nodes: classify, strategy, direct_answer, web_search, decompose, multi_hop, tools, simple_retrieve, grade, generate, rewrite, fallback, abort

Stage	Decides	Options	Condition fn
classify	Do we need the corpus at all?	direct · retrieve · web_search	route_condition
strategy	How should we retrieve?	decompose · multi_hop · tools · simple	strategy_condition ← choose_strategy
grade	Is evidence good enough?	generate · rewrite · fallback	grade_condition

IMPORTANT IMPLEMENTATION DETAILS

- Strategy nodes reuse Phase 4–6 graphs with `skip_router=True`; nested token emission is suppressed so the outer graph owns the streamed final answer.
- After `rewrite`, the edge is `rewrite → grade` (rewrite re-retrieves internally). This differs from standalone Crag, where `rewrite` loops back to a named `retrieve` node.
- Tools with empty graded docs but a prior tool answer: `grade_condition` routes to `generate` to keep that answer instead of forcing web fallback.
- Strategy prompt lives in `src/agents/orchestrator.py` (`STRATEGY_PROMPT`), not `src/prompts.py`.

7. Decision chains the agents use

Graphs call small LangChain structured-output chains. These are the “brains” inside nodes.

Module	Function / chain	Produces
agents/router.py	classify_query / router chain	route $\in$ {direct, retrieve, web_search} + reason
agents/grader.py	grade_documents	Keep chunks with relevant and score $\geq$ threshold
agents/query_rewriter.py	rewrite_query	Rewritten search query + reason
agents/decomposer.py	decompose_query	1–5 sub_queries + reasoning
agents/multi_hop.py	analyze_question , reflect_on_hop	First hop plan; continue vs synthesize
agents/orchestrator.py	choose_strategy / normalize_strategy	decompose multi_hop tools simple
agents/followups.py	generate_follow_ups	Up to 3 grounded follow-up questions

7.1 Strategy selection rubric (agentic)

Strategy	When the orchestrator should pick it	Example question
decompose	Comparisons / multiple distinct entities known upfront	Compare naive, advanced, and modular RAG
multi_hop	Answer needs fact A before you can search for fact B	What fallback does CRAG use when retrieval fails?
tools	Mix of retrieval + math/web/function calling	What is Self-RAG and what is 20×30?
simple	Single-topic factual lookup	What is Self-RAG?

7.2 Prompt catalog

Most prompts live in `src/prompts.py`: `ROUTER_PROMPT` , `RAG_PROMPT` , `DIRECT_PROMPT` , `WEB_SEARCH_PROMPT` , `GRADER_PROMPT` , `QUERY_REWRITE_PROMPT` , `DECOMPOSE_PROMPT` , `SYNTHESIS_PROMPT` , `MULTI_HOP_ANALYZE_PROMPT` , `HOP_REFLECT_PROMPT` , `FOLLOWUP_PROMPT` . Strategy selection uses `STRATEGY_PROMPT` in the orchestrator.

8. Ingestion & retrieval pipeline

8.1 Ingestion (before any query)

Entry: `python -m src.ingestion.ingest --source data/sample_docs` · Use `--force` to rebuild.

```
ingest()
→ optional reset_collection()
→ load_pdfs(source_dir)          # PyMuPDF
→ per PDF: chunk_documents(...)
    cleanse (headers/footers/boilerplate/hyphenation/irrelevant sections)
    chunk:
        • section_parent_child (default): TOC → regex headings → fixed fallback
        • fixed: RecursiveCharacterTextSplitter
→ save_parents(all_parents) → PARENT_STORE_PATH (JSON)
→ assign chunk_content_id → Chroma upsert (delete overlapping ids first)
→ invalidate_bm25_cache() + flush_answer_cache()
```

Module	Owns
ingestion/cleanse.py	Noise removal gated by <code>CLEANSE_*</code>
ingestion/chunking.py	Parent/child or fixed chunking
ingestion/parent_store.py	Parent JSON + <code>expand_children_to_parents</code> at retrieve time
ingestion/ingest.py	Embeddings (<code>OpenAIEmbeddings</code>), Chroma persistent/HTTP, CLI

Compose uses `CHROMA_MODE=http` ; local dev defaults to persistent SQLite under `data/chroma_db` . Embeddings always stay on OpenAI even if chat falls back to Groq.

8.2 Retrieval (shared by all modes)

Entry: `retrieve(query, top_k=None)` in `src/retrieval/retriever.py` .

```
query
→ over-fetch candidate_k
    hybrid (default): dense similarity + BM25 → Reciprocal Rank Fusion (_rrf_fuse)
    or mmr / similarity
→ optional cross-encoder rerank
    nvidia (NeMo API, circuit breaker "nvidia_rerank")
    or flashrank (local ONNX)
→ truncate toward top_k
→ optional expand_children_to_parents (cap parent_max_chars)
→ documents for grading / generation / citations
→ _log_retrieval
```

Knob	Default role
RETRIEVAL_CANDIDATE_K	Over-fetch pool before rerank (often 20)
RETRIEVAL_TOP_K	Final chunks to the LLM (often 6)
RETRIEVAL_SEARCH_TYPE=hybrid	Dense + BM25 + RRF
RERANK_ENABLED / RERANK_PROVIDER	nvidia or flashrank
EXPAND_TO_PARENT=true	Return parent sections for richer context

MULTI-WORKER NOTE

BM25 is an in-process index rebuilt when collection count changes — it is **not** shared across API workers.

9. Generation, citations, follow-ups, streaming

9.1 Generation chains

LCEL chains in `src/chains/generation.py`: `rag_chain`, `direct_chain`, `synthesis_chain`, `web_search_chain`. Pattern: `prompt | get_llm() | StrOutputParser`.

9.2 LLM client — `src/llm.py`

- **Primary:** OpenAI ChatOpenAI with hard `timeout`, `max_retries`, `max_tokens`.
- **Fallback:** if `LLM_FALLBACK_ENABLED` and `GROQ_API_KEY` are set, `FallbackChatModel` wraps both providers with `with_fallbacks`, and mirrors `with_structured_output` / `bind_tools`.
- `reset_llm_provider` / `get_llm_provider` ContextVars attribute spend correctly when fallback fires.

9.3 Citations — `src/retrieval/citations.py`

Builds Citation objects (`src/schemas.py`): `index`, `chunk_id`, `source`, `page`, `section`, `snippet`, `score`.
Helpers: `docs_to_citations`, `docs_to_sources`, `docs_to_context`, `build_response`.

9.4 Follow-ups — `src/agents/followups.py`

After every successful (non-abort) run, `generate_follow_ups` proposes up to three questions grounded in retrieved snippets (or the answer text as fallback). Failures are swallowed so follow-up generation never fails a query.

9.5 Streaming internals — `src/streaming.py`

- `use_emitter` / `emit_step` / `emit_token` — ContextVar-based event bus.
- `stream_text` / `stream_llm_message` — token streaming for final answers.
- `run_graph_streaming(graph, initial_state)` — `stream_mode="values"`; emits new `steps` as nodes complete.
- `suppress_token_emit` — used inside nested strategy subgraphs that regenerate later.
- `CancelledRun` — raised when the API cancels a stream on disconnect/timeout.

9.6 Schemas — `src/schemas.py`

Public response shape is `AgentResponse` (includes `error_code`). Agent structured models (`RouteDecision`, `DocumentGrade`, `StrategyChoice`, etc.) live near their agents.

10. Production envelope

Everything outside the graphs that makes the system safe to expose.

Concern	Mechanism	Where
Auth	API key via <code>X-API-Key</code> ; constant-time <code>secrets.compare_digest</code> ; mandatory if <code>ENVIRONMENT=production</code> or <code>REQUIRE_API_KEY</code>	<code>api/security.py</code>
Per-client rate limit	Sliding 60s window; Redis when <code>RATE_LIMIT_BACKEND=auto redis</code> ; else in-memory deque	<code>api/rate_limit.py</code>
Process cost budget	QPM + tokens/min + tokens/hour via <code>CostGuardrails</code>	<code>guardrails.py</code> (inside runner)
Privacy	PII/PHI detect; input/output modes <code>redact block off</code> ; PHI opt-in	<code>privacy.py</code>
Answer cache	Redis key <code>rag:v1:{mode}:{sha256(norm(q))}</code> ; skip when memory history active; flush on re-ingest	<code>cache/redis_cache.py</code>
Idempotency	<code>Idempotency-Key</code> + body hash on sync <code>/query</code> ; TTL <code>idempotency_ttl_seconds</code>	server + Redis
Circuit breakers	Named breakers for <code>nvidia_rerank</code> and <code>web_search</code>	<code>resilience/circuit_breaker.py</code>
Node output gates	Quarantine/abort on tool sentinels, invalid routes, empty critical answers; <code>error_code=node_gate_abort</code>	<code>resilience/node_gate.py</code>
Concurrency	Semaphore + acquire timeout → 503	<code>api/server.py</code>
Metrics	Prometheus at <code>GET /metrics</code> (protected by default)	<code>api/metrics.py</code>
Health	<code>/health</code> liveness; <code>/health/ready</code> deep checks (protected by default)	<code>api/health.py</code>
Tracing	Optional LangSmith via bootstrap	<code>observability / bootstrap</code>
Memory	Compact packing + optional Supabase	<code>memory/*</code>

MULTI-WORKER RULE

Before raising `API_WORKERS` above 1, run Redis and set `RATE_LIMIT_BACKEND=redis` (compose already does). In-memory limits are per-process and under-count under load balancing. Production validation refuses to boot otherwise.

10.1 Health checks — `deep_health()`

Check	Required?	Meaning
chroma	yes	Vector store reachable / countable
openai	yes	API key present (no live call)
data_dir	yes	Persist dir or HTTP Chroma URL
redis	optional	Skipped if cache off and rate backend is memory
groq / supabase / nvidia	optional	Only when those features are enabled

Overall: any required `error` → unhealthy; any `degraded` → degraded; else healthy. Ready endpoint returns 200 for healthy/degraded, else 503.

10.2 Metrics of note

`rag_requests_total`, `rag_request_latency_seconds`, `rag_cache_events_total`, `rag_llm_fallback_total`, `rag_rate_limit_total`, `rag_node_gate_total`, `rag_capacity_rejections_total`, `rag_tokens_total`, `rag_cost_usd_total`.

10.3 Evaluation

- **Golden retrieval:** `data/eval/golden_qa.json` + `python -m src.evaluation.retrieval_metrics` (use `--offline` in CI).
- **LLM-as-judge:** `python -m src.evaluation.evaluate_all_modes` → faithfulness / relevance / context precision.
- Nightly: `.github/workflows/nightly-eval.yml`.

11. API surface, clients & deploy

11.1 HTTP API

Method	Path	Auth	Rate limit	Purpose
GET	<code>/health</code>	public	no	Liveness
GET	<code>/health/ready</code>	gated*	no	Readiness (deep)
GET	<code>/metrics</code>	gated*	no	Prometheus scrape
GET	<code>/modes</code>	yes†	no	Mode labels
POST	<code>/query</code>	yes†	yes	Sync JSON answer
POST	<code>/query/stream</code>	yes†	yes	SSE progressive answer

* Protected by default via `PROTECT_METRICS_ENDPOINT` / `PROTECT_READINESS_ENDPOINT` (compose may relax on the internal network).

† Auth enforced when `REQUIRE_API_KEY=true` or `ENVIRONMENT=production`.

```

POST /query
{
  "question": "What is Self-RAG?",
  "mode": "agentic",
  "session_id": "optional-id",
  "use_memory": true,
  "chat_history": [
    {"role": "user", "content": "..."},
    {"role": "assistant", "content": "..."}
  ]
}

```

11.2 React frontend

- Dev: Vite on :5173; prod image: nginx on :8080 proxying /api → API.
- Primary path: streamQuery in frontend/src/api/client.ts.
- useChat wires SSE → steps, tokens, follow-ups, citations, traces.
- Local chatStore holds history when Supabase is unavailable.

11.3 Docker Compose services

Service	Port	Role
redis	6379 (internal)	Cache, shared rate limits, idempotency
chroma	8001→8000 (internal)	Vector store (HTTP)
agentic-rag	8000	FastAPI app
frontend	8080	nginx + React; can inject API key server-side
prometheus	9090	Optional metrics store
grafana	3000	Optional “Agentic RAG Overview” dashboard

```

# Core stack
cp .env.production.example .env.production # fill secrets
docker compose up -d

# Optional observability
docker compose up prometheus grafana -d

```

Bare-metal alternative: `deploy.sh` loads `.env.production`, validates secrets, installs deps, runs uvicorn. Prefer compose for the full stack.

12. Configuration map

All settings load via pydantic-settings in `src/config.py` from environment / `.env`.

Group	Key variables
Environment	ENVIRONMENT , API_WORKERS
LLM	OPENAI_API_KEY , OPENAI_MODEL , OPENAI_EMBEDDING_MODEL , timeouts/retries/ MAX_OUTPUT_TOKENS
Fallback	LLM_FALLBACK_ENABLED , GROQ_API_KEY , GROQ_MODEL
Vector store	CHROMA_MODE (persistent http), host/port, COLLECTION_NAME
Chunking	CHUNKING_STRATEGY , child/parent sizes, EXPAND_TO_PARENT , PARENT_STORE_PATH
Cleanse	CLEANSE_ENABLED and CLEANSE_* toggles
Retrieval	RETRIEVAL_TOP_K , RETRIEVAL_CANDIDATE_K , RETRIEVAL_SEARCH_TYPE , RRF/MMR knobs
Rerank	RERANK_ENABLED , RERANK_PROVIDER , RERANK_MODEL , NVIDIA_API_KEY
Agent	MAX_RETRIEVAL_RETRIES , GRADER_RELEVANCE_THRESHOLD , MAX_MULTI_HOP_STEPS
API / security	API_KEY , REQUIRE_API_KEY , CORS_ORIGINS , TRUSTED_HOSTS , protect metrics/ready
Rate / cost	MAX_QUERIES_PER_MINUTE* , MAX_TOKENS_* , RATE_LIMIT_BACKEND , TRUST_PROXY_HEADERS
Request handling	REQUEST_TIMEOUT_SECONDS , STREAM_TIMEOUT_SECONDS , concurrency limits
Cache / Redis	CACHE_ENABLED , REDIS_URL , CACHE_TTL_SECONDS , idempotency TTL
Resilience	CIRCUIT_BREAKER_*
Privacy	PRIVACY_INPUT_MODE , PRIVACY_OUTPUT_MODE , PRIVACY_DETECT_PHI
Memory	MEMORY_ENABLED , MEMORY_MAX_TURNS , MEMORY_RECENT_EXCHANGES , MEMORY_ANSWER_MAX_CHARS , MEMORY_MAX_OLDER_QUERIES , SUPABASE_*
Observability	LANGSMITH_* , QUALITY_GUARDRAILS_ENABLED

13. File ownership quick reference

Path	Owns
src/runner.py	Mode dispatch, lifecycle, cache/memory/follow-ups/streaming orchestration
src/graph/*_graph.py	Per-mode LangGraph topologies + ask_* entrypoints
src/rag/baseline.py	Phase 1 fixed pipeline
src/agents/*	Structured LLM decision chains + follow-ups
src/memory/*	History packing + Supabase persistence
src/retrieval/*	Hybrid search, rerank, citations
src/ingestion/*	Cleanse, chunk, parent store, Chroma upsert
src/tools/all_tools.py	retrieve_docs, web_search, calculator
src/llm.py	OpenAI primary + optional Groq fallback
src/streaming.py	SSE emitter, tokens, graph step streaming
src/cache/redis_cache.py	Answer cache + idempotency helpers
src/resilience/*	Circuit breakers + node gates
src/api/server.py	HTTP routes, history resolve/persist, response models
src/api/metrics.py	Prometheus instrumentation
frontend/	React chat UI (SSE, citations, follow-ups, traces, memory toggle)
monitoring/	Prometheus scrape config + Grafana dashboard
data/eval/golden_qa.json	Retrieval golden set

14. Optional features & known gaps

#	Item	Default / reality
1	Idempotency	Only on /query , not stream
2	Quality guardrails	Off by default (extra LLM cost)
3	Answer cache	Often off in local; disabled when conversation history active
4	Node-gate abort plumbing	Fullest in CRAG / tools / agentic; thinner in router / decompose / multi_hop
5	Streamlit / CLI	Bypass FastAPI auth, rate limits, concurrency, idempotency
6	PHI detection	Opt-in via <code>PRIVACY_DETECT_PHI</code>
7	Rerank / Redis / Supabase / Groq / LangSmith	All optional; degrade gracefully when unset
8	Agentic rewrite loop	Re-grades strategy outputs; CRAG rewrite re-retrieves via named retrieve node
9	Sync timeout workers	Thread may continue briefly after 504; mitigated by provider hard timeouts
10	BM25 across workers	Per-process; not shared

15. Tests that document behavior

Prefer reading these tests when a behavior is unclear — they encode the contract.

File	Documents
tests/test_api.py	Health, metrics, request-id, idempotency, auth, query + history, validation, 429
tests/test_production_safety.py	Metrics/ready gates, CORS/API key/worker validation, capacity 503, rate-limit prune
tests/test_runner.py	Short input, PII redact/block, unknown mode, memory uses sanitized question
tests/test_streaming.py	Token emit, SSE shape, privacy error event, cancellation
tests/test_cache.py	Key normalization, skip with memory, hit skips dispatch, Redis down
tests/test_rag_graphs.py	Graphs compile, agentic edges through grade, baseline context_docs
tests/test_node_gate.py	Quarantine/abort contracts for tools/routes/answers
tests/test_circuit_breaker.py	Open after threshold, recover
tests/test_retrieval.py	format/citations/RRF/rerank providers
tests/test_memory.py	History packing / augment
tests/test_privacy.py	PII/PHI policy matrix
tests/test_followups.py	Exactly 3; runner attaches
tests/test_llm.py	Groq fallback + structured/tools bind
tests/test_tools.py	Calculator sandbox
tests/test_chunking.py / test_cleanse.py	Ingest structure
tests/test_golden_offline.py	Golden QA thresholds
tests/load/locustfile.py	Load against API

16. Worked traces

Re-run these with `-v` to see `steps` fill in live.

16.1 Simple fact — `baseline` vs `crag`

```
python -m src.cli ask "What is Self-RAG?" --mode baseline -v
python -m src.cli ask "What is Self-RAG?" --mode crag -v
```

WHAT TO NOTICE

Baseline: one retrieve → generate. CRAG: retrieve → grade each chunk → maybe rewrite → generate (or web fallback if the corpus is weak).

16.2 Comparison — `decompose`

```
python -m src.cli ask "Compare naive RAG, advanced RAG, and modular RAG" --mode decompose -v
```

WHAT TO NOTICE

`sub_queries` appear; parallel `retrieve_sub` workers; one synthesis.

16.3 Sequential dependence — `multi_hop`

```
python -m src.cli ask "What fallback does CRAG use when retrieval fails?" --mode multi_hop -v
```

WHAT TO NOTICE

Hop 1 finds CRAG; reflect builds hop 2 query about fallback; synthesize.

16.4 Full orchestrator — `agentic`

```
python -m src.cli ask "Compare RAG vs Agentic RAG; what is Self-RAG grading?" --mode agentic -v
```

WHAT TO NOTICE

Steps show Router → retrieve, then Strategy: `decompose|...`, then grade, then generate. Same question through baseline will be shallower.

16.5 Memory follow-up (API)

```
curl -s -X POST http://localhost:8000/query \
-H "Content-Type: application/json" \
-d '{
  "question": "What about the second approach?",
  "mode": "agentic",
  "use_memory": true,
  "chat_history": [
    {"role": "user", "content": "Compare naive and advanced RAG"},
    {"role": "assistant", "content": "Naive RAG retrieves once... Advanced RAG adds rewriting..."}
  ]
}' | jq '.answer,.steps'
```

WHAT TO NOTICE

Without history, “the second approach” is ambiguous. With memory packing, the runner injects prior Q/A so the agent resolves “second” to advanced RAG.

16.6 HTTP smoke

```
uvicorn src.api.server:app --reload --port 8000

curl -s http://localhost:8000/health
curl -s -X POST http://localhost:8000/query \
  -H "Content-Type: application/json" \
  -d '{"question":"What is Self-RAG?","mode":"agentic"}' | jq '.answer,.steps,.follow_ups'

curl -N http://localhost:8000/query/stream \
  -H "Content-Type: application/json" \
  -d '{"question":"What is Self-RAG?","mode":"agentic"}'
```

Agentic RAG — System Reference Manual (Deep Dive) · August 2026

Canonical code entry points: `src/runner.py`, `src/graph/`, `src/memory/`, `src/api/server.py`. Companion docs: `README.md`, `docs/ROADMAP.md`, `docs/PRODUCTION.md`, `docs/GUARDRAILS.md`, `docs/LANGCHAIN_STACK.md`, `docs/PRIVACY_COMPLIANCE.md`.

If code and this manual diverge, trust the code — then update this file.